3. Concurrency

Computer and Virtual Technologies Engineering
Software Engineering

Operating Systems

2025-2026

Universidad LOYOLA

Contents

1. Introduction
2. Race condition
3. The critical section problem
4. Synchronization hardware
5. Mutex locks
6. Semaphores

Contents

1. Introduction
2. Race condition
3. The critical section problem
4. Synchronization hardware
5. Mutex locks
6. Semaphores

Introduction to concurrency

The central themes of **operating system design** are concerned with the management of processes and threads:

- **Multiprogramming**: the management of multiple processes within a uniprocessor system.
- **Multiprocessing**: the management of multiple processes within a multiprocessor.
- **Distributed processing**: the management of multiple processes executing on multiple, distributed computer systems.

Concurrency arises in three different contexts:

- **Multiple applications**: processing time is dynamically shared among a number of active applications.
- **Structured applications**: some applications can be programmed as a set of concurrent processes.
- **Operating system structure**: operating systems are themselves often implemented as a set of processes or threads.

Introduction to concurrency

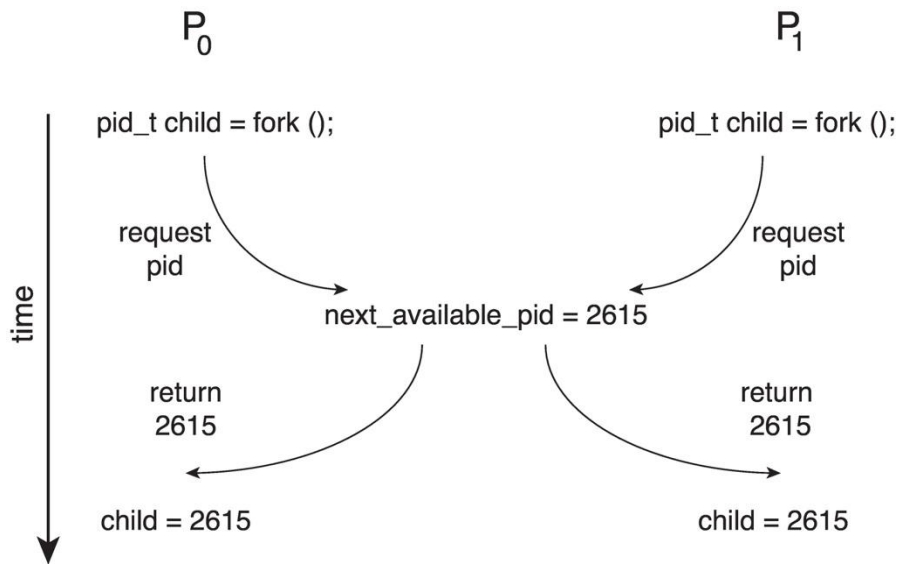
- The basic requirement for support of **concurrent processes** is the ability to enforce **mutual exclusion**.
- **Mutual exclusion:** the ability to exclude all other processes from a course of action while one process is granted that ability.
- It is a problem of resource sharing.
- **Critical section:** a section of code within a process that requires access to shared resources, and that must not be executed while another process is in a corresponding section of code.
- A successful solution to the problem has to avoid:
  - **Deadlocks**, a situation in which two or more processes are unable to proceed because each is waiting for one of the others to do something.
  - **Livelocks**, a situation in which two or more processes continuously change their states in response to changes in the other process without doing any useful work.

Contents

1. Introduction
2. Race condition
3. The critical section problem
4. Synchronization hardware
5. Mutex locks
6. Semaphores

Race condition

- Processes P_0 and P_1 are creating child processes using the `fork()` system call.
- Race condition on kernel variable `next_available_pid` which represents the next available process identifier (pid).
- Unless there is a mechanism to prevent P_0 and P_1 from accessing the variable. `next_available_pid` the same pid could be assigned to two different processes!



Race condition

```
/* Thread 0 */

temp = count
count = temp + 1
```

```
/* Thread 1 */

temp = count
count = temp + 1
```

Execution:
T0_line1
T1_line1
T1_line2
T0_line2

Race condition

```
/* Thread 0 */                    /* Thread 1 */

count = count + 1                 count = count + 1
```

These operations are translated into machine language before execution.
In machine language the update takes more steps: read, write…

Contents

1. Introduction
2. Race condition
3. The critical section problem
4. Synchronization hardware
5. Mutex locks
6. Semaphores

Critical section problem

- Consider system of ***n*** processes {$p_0, p_1, \ldots p_{n-1}$}.
- Each process has **critical section** segment of code.
  - Process may be changing common variables, updating table, writing file, etc.
  - When one process in critical section, no other may be in its critical section.
- ***Critical section problem*** is to design protocol to solve this.
- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section.**

Critical section problem

- General structure of the process p_i.

```
while (true) {
        entry section
            critical section
        exit section
            remainder section
}
```

Critical section problem

Requirements for solution to critical-section problem:

- **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
- **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the process that will enter the critical section next cannot be postponed indefinitely.
- **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.
  - Assume that each process executes at a nonzero speed.
  - No assumption concerning **relative speed** of the ***n*** processes.

Contents

1. Introduction
2. Race condition
3. The critical section problem
4. Synchronization hardware
5. Mutex locks
6. Semaphores

Synchronization hardware

- Many systems provide hardware support for implementing the critical section code.
- **Atomic operation**: a function or action implemented as a sequence of one or more instructions that appears to be indivisible. No other process can see an intermediate state or interrupt the operation. The sequence of instruction is guaranteed to execute as a group, or not execute at all, having no visible effect on system state. Atomicity guarantees isolation from concurrent processes.
- Implementations:
  - **Software schemes**: Peterson's algorithms.
    - This would entail a substantial processing overhead.
    - Peterson’s solution is not guaranteed to work on modern architectures.

  - **Hardware-supported schemes**:
    - *compare&swap* instruction.
    - Inhibit interrupts for the duration of *wait* and *signal* operations.

Interrupt disabling

- In a uniprocessor system, concurrent processes cannot have overlapped execution. They can only be interleaved.
- A process will continue to run until it invokes an OS service or until it is interrupted. Therefore, to guarantee mutual exclusion, it is sufficient to prevent a process from being interrupted.
- This capability can be provided in the form of primitives defined by the OS kernel for disabling and enabling interrupts.
- Problems:
  - Efficiency of execution could be degraded because the processor is limited in its ability to interleave processes.
  - This approach will not work in a multiprocessor architecture.

```
while (true){
     /*disable interrupts*/
     /*critical section*/
     /*enable interrupts*/
     /*remainder*/
}
```

compare&swap instruction

- Processor designers have proposed several machine instructions that carry out two actions atomically, such as reading and writing or reading and testing, of a single memory location with one instruction. During execution of the instruction, access to the memory location is blocked for any other instruction referencing that location.
- The *compare&swap* instruction:

```
int compare_and_swap (int *word, int testval, int newval){
    int oldval;
    oldval = *word;
    if (oldval == testval)
        *word = newval;
    return oldval;
}
```

- This version of the instruction checks a memory location *word against a test value.
- The entire *compare&swap* function is carried out atomically. It is not subject to interruption.

compare&swap instruction

Hardware support for mutual exclusion

```
const int n = /* number of processes */
int bolt;
void P(int i){
    while (true){
        while (compare_and_swap(&bolt,0,1)==1); /* do nothing */
        /* critical section */
        bolt = 0;
        /* remainder */
    }
}
void main(){
    bolt = 0;
    parbegin(P(1),P(2),…,P(n));
}
```

The only process that may enter its critical section is one that finds bolt equal to 0

Exercise

Another atomic machine instruction that supports mutual exclusion that is often mentioned in the literature is the *test&set* instruction, defined as follow:

```
boolean test_and_set (boolean *target){
    boolean rv = *target;
    *target = true;

    return rv;
}
```

Define a procedure that uses the *test&set* instruction to solve the problem of mutual exclusion.

Solution

```
do{
  while(test_and_set(&lock)); /* do nothing*/

  /* critical section */;

  lock = false;

  /* remainder section */

} while (true);
```

Properties of *compare&swap*, *test&set*

- Advantages:
  - Any number of processes.
  - Simple and easy to verify.
  - Multiple critical sections.

- Disadvantages:
  - Busy waiting is employed: while a process is waiting for access to a critical section, it continues to consume processor time.
  - Starvation is possible: the selection of a waiting process is arbitrary.
  - Deadlock is possible.

Properties of *compare&swap*, *test&set*

- Advantages:
  - Any number of processes.
  - Simple and easy to verify.
  - Multiple critical sections.

- Disadvantages:
  - Busy waiting is employed: while a process is waiting for access to a critical section, it continues to consume processor time.
  - Starvation is possible: the selection of a waiting process is arbitrary.
  - Deadlock is possible.

- Other mechanisms are needed to solve the mutual exclusion problem (such as semaphores).
- Typically, instructions such as *compare&swap* are used as **building blocks for other synchronization tools.**
- One tool is an **atomic variable** that provides *atomic* (uninterruptible) updates on basic data types such as integers and booleans.

Atomic variables

Example:

- Let `sequence` be an atomic variable.
- Let `increment()` be operation on the atomic variable `sequence`.
- The command `increment(&sequence);` ensures `sequence` is incremented without interruption.

```
void increment(atomic_int *v)
{
        int temp;
        do {
                temp = *v;
        }
        while (temp != (compare_and_swap(v,temp,temp+1));
}
```

Contents

1. Introduction
2. Race condition
3. The critical section problem
4. Synchronization hardware
5. Mutex locks
6. Semaphores

Mutex locks

- Previous solutions are complicated and generally inaccessible to application programmers.
- OS designers build software tools to solve critical section problem.
- Simplest is mutex lock.
  - Boolean variable indicating if lock is available or not.
- Protect a critical section by:
  - First `acquire()` a lock.
  - Then `release()` the lock.
- Calls to **`acquire()`** and **`release()`** must be **atomic.**
  - Usually implemented via hardware atomic instructions such as compare&swap.
- But this solution requires **busy waiting.**
  - This lock therefore called a **spinlock.**

Solution to critical section problem using Mutex locks

```
while (true) {

      acquire lock

          critical section

      release lock

remainder section
}
```

Contents

1. Introduction
2. Race condition
3. The critical section problem
4. Synchronization hardware
5. Mutex locks
6. Semaphores

Semaphores

The first major advance in dealing with the problems of concurrent processes came in 1965 with Dijkstra's treatise. Two or more processes can cooperate by means of simple signals, such that a process can be **forced** to **stop** at a specified place **until receiving** a **signal**.

- **Semaphore**: an integer value used for signaling among processes. Only three operations may be performed on a semaphore, all of which are atomic: initialize, decrement and increment.
- **Binary semaphore**: a semaphore that takes on only the values 0 and 1.
- A **mutex lock** is similar to binary semaphore but the process that locks the mutex (sets the value to 0) must be the one to unlock it (sets the value to 1).

Semaphores

- Synchronization tool to synchronize the activities of processes.
- Semaphore ***S*** – integer variable.
- Can only be accessed via two indivisible (atomic) operations.
  - **wait()** and **signal()**
  - Originally called **P()** and **V()**
- Definition of the **wait()** operation.

```
wait(S){
    while (S <= 0)
        ; // busy wait
    S--;
}
```

- Definition of the **signal()** operation.

```
signal(S){
    S++;
}
```

Semaphore usage example

Consider P_1 and P_2 that with two statements S_1 and S_2 and the requirement that S_1 to happen before S_2

Create a semaphore "**synch**" initialized to 0.

```
P1:
    S1;
    signal(synch);
P2:
    wait(synch);
    S2;
```

Mutual exclusion using semaphores

```
const int n = N; /*Number of processes*/
semaphore s = 1;

void P(int i){

   while(true){
      wait(s);
      /*critical section*/
      signal(s);
      /*remainder*/
   }
}

void main(){

/* parbegin suspends the execution of the main program, initiates
concurrent execution of processes P(1),P(2),…,P(n) and when all
processes have terminated, resume the main program.*/

parbegin(P(1),P(2),…,P(N));

}
```

Semaphore implementation with no busy waiting

- With each semaphore there is an associated waiting queue.
- Each entry in a waiting queue has two data items:
  - Value (of type integer).
  - Pointer to next record in the list.
- Two operations:
  - **block** – place the process invoking the operation on the appropriate waiting queue.
  - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue.

Semaphore implementation with no busy waiting

```
typedef struct{
  int value;
  struct process *list;
}semaphore;

wait(semaphore *S){
    S->value--;
    if (S->value < 0){
        add this process to S->list;
        block();
    }
}
```

```
signal(semaphore *S){
    S->value++;
    if (S->value <= 0){
        remove a process P from S->list;
        wakeup(P);
    }
}
```

Strong/weak semaphores

A queue is used to hold processes waiting on the semaphore.

Strong Semaphores

- The process that has been blocked the longest is released from the queue first (FIFO)

Weak Semaphores

- The order in which processes are removed from the queue is not specified

Universidad
LOYOLA

Universidad
LOYOLA